

What is Inheritance?

Inheritance lets a child class use the properties and methods of a parent class — like a son inheriting money or habits from a father.

```
class Parent:
```

```
    def speak(self):  
        print("Speaking...")
```

```
class Child(Parent): # Inheriting from Parent
```

```
    def walk(self):  
        print("Walking...")
```

```
c = Child()
```

```
c.speak() # From Parent
```

```
c.walk() # From Child
```

1. Single Inheritance

One child inherits from one parent.

```
class Parent:
```

```
    def show(self):  
        print("I am the Parent")
```

```
class Child(Parent):
```

```
    def display(self):  
        print("I am the Child")
```

```
c = Child()
```

```
c.show() # From Parent
```

```
c.display() # From Child
```

2. Multiple Inheritance

One child inherits from more than one parent.

```
class Father:
```

```
    def skill1(self):  
        print("Knows driving")
```

```
class Mother:
```

```
    def skill2(self):  
        print("Knows cooking")
```

```
class Child(Father, Mother):
```

```
    pass
```

```
c = Child()
```

```
c.skill1()
```

```
c.skill2()
```

3. Multilevel Inheritance

Grandchild inherits from child who inherits from parent.

```
class Grandparent:
```

```
    def house(self):  
        print("Has a big house")
```

```
class Parent(Grandparent):
```

```
    def car(self):  
        print("Has a car")
```

```
class Child(Parent):
```

```
    def bike(self):  
        print("Has a bike")
```

```
c = Child()
c.house()
c.car()
c.bike()
```

4. Hierarchical Inheritance

One parent, multiple children.

```
class Parent:
    def speak(self):
        print("Parent speaks")
```

```
class Child1(Parent):
    def play(self):
        print("Child1 plays")
```

```
class Child2(Parent):
    def dance(self):
        print("Child2 dances")
```

```
c1 = Child1()
c2 = Child2()
c1.speak()
c2.speak()
```

5. Hybrid Inheritance

Combination of more than one type. Python handles this using MRO (Method Resolution Order).

```
class A:
    def show(self):
        print("A class")
```

```
class B(A):
```

```
    pass
```

```
class C(A):
```

```
    def show(self):
```

```
        print("C class")
```

```
class D(B, C): # Inherits from B and C (Hybrid)
```

```
    pass
```

```
d = D()
```

```
d.show()
```

Method Resolution Order (MRO)

Definition

The **Method Resolution Order (MRO)** in Python is the sequence in which Python searches for a method or attribute in a hierarchy of classes during inheritance.

It ensures that when multiple classes are involved in inheritance, Python consistently determines which method or attribute should be executed first.

Python follows the **C3 Linearization Algorithm** to compute this order.

Why is MRO Needed?

When using **multiple inheritance**, a child class may inherit methods with the same name from different parent classes. MRO helps Python decide the order in which these methods are searched and executed.

For example:

```
class A:
```

```
    def show(self):
```

```
        print("Class A")
```

```
class B(A):
```

```
pass
```

```
class C(A):  
    def show(self):  
        print("Class C")
```

```
class D(B, C):  
    pass
```

```
d = D()  
d.show()
```

Here, both class A and class C define the method show(). Python resolves this conflict using MRO. The output will be:

Class C

Checking MRO in Python

You can view the MRO of a class in two ways:

1. `ClassName.__mro__`
2. `ClassName.mro()`

Example:

```
print(D.mro())
```

Output:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

This means that Python will search for methods in the order:

D → B → C → A → object

Rules of MRO (C3 Linearization)

1. **Child class before parent class:** The search always starts from the child class.
2. **Left-to-right order:** In multiple inheritance, classes are checked from left to right in the parent list.
3. **No duplicate visits:** If a class is already visited, it is not searched again.

4. **Consistency:** The algorithm ensures that all parent classes are considered in a consistent and predictable order.
-

The Diamond Problem and MRO

The "diamond problem" occurs when a class inherits from two classes that both inherit from a common parent.

Example:

```
class A:
```

```
    def show(self):  
        print("Class A")
```

```
class B(A):
```

```
    def show(self):  
        print("Class B")
```

```
class C(A):
```

```
    def show(self):  
        print("Class C")
```

```
class D(B, C):
```

```
    pass
```

```
d = D()
```

```
d.show()
```

Output:

Class B

MRO for class D is:

$D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$

Here, even though both B and C inherit from A, Python ensures that A is called only once and in a consistent order, avoiding ambiguity.

Key Points

- MRO defines the order of method lookup in classes with inheritance.
- It uses the **C3 linearization algorithm**.
- It solves problems caused by multiple inheritance, such as the diamond problem.
- The `super()` function works according to MRO.

Type Structure & Example Classes

Type	Structure	Example Classes
Single	$A \rightarrow B$	Parent $\rightarrow$ Child
Multiple	$A, B \rightarrow C$	Father, Mother $\rightarrow$ Child
Multilevel	$A \rightarrow B \rightarrow C$	Grandparent $\rightarrow$ Parent $\rightarrow$ Child
Hierarchical	$A \rightarrow B, C$	Teacher $\rightarrow$ Student1, Student2
Hybrid	Combo of above	$A \rightarrow B, C \rightarrow D$

`super()` in Python Inheritance

Definition

The `super()` function in Python is used to call a method from the **parent class** (or the next class in the **MRO – Method Resolution Order**) inside a child class.

It allows you to extend or modify the behavior of inherited methods without completely overriding them.

Why is `super()` Needed?

1. To **reuse parent class code** instead of rewriting it.
2. To ensure **proper method execution order** in multiple inheritance (using MRO).
3. To make classes more maintainable and scalable.

Basic Example

```
class Parent:
```

```
    def display(self):
```

```
print("This is the Parent class")
```

```
class Child(Parent):  
    def display(self):  
        super().display() # Calling Parent's method  
        print("This is the Child class")
```

```
c = Child()
```

```
c.display()
```

Output:

This is the Parent class

This is the Child class

Here, `super().display()` calls the parent class method, then the child's method adds extra functionality.

Use in Constructors (__init__)

```
class Parent:  
    def __init__(self, name):  
        self.name = name  
        print("Parent constructor")  
  
class Child(Parent):  
    def __init__(self, name, age):  
        super().__init__(name) # Calls Parent constructor  
        self.age = age  
        print("Child constructor")
```

```
c = Child("Atharva", 20)
```

Output:

Parent constructor

Child constructor

Without `super()`, the parent's `__init__` would not run automatically.

`super()` with Multiple Inheritance

```
class A:
```

```
    def show(self):  
        print("A")
```

```
class B(A):
```

```
    def show(self):  
        super().show()  
        print("B")
```

```
class C(A):
```

```
    def show(self):  
        super().show()  
        print("C")
```

```
class D(B, C):
```

```
    def show(self):  
        super().show()  
        print("D")
```

```
d = D()
```

```
d.show()
```

Output (MRO applied):

A

C

B

D

Explanation:

Step 1: Check the MRO of D

```
print(D.mro())
```

Output:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

So, the search order is:

D → B → C → A → object

Step 2: Method Calls

- d.show() starts in D.
- Inside D.show(), we call super().show(). According to MRO, after D, the next is B.
- B.show() runs. Inside B.show(), it again calls super().show(). After B in MRO comes C.
- C.show() runs. Inside C.show(), it again calls super().show(). After C in MRO comes A.
- A.show() runs. Inside A.show(), there is no super(), so it just prints "A".

Now the stack unwinds:

- C.show() prints "C" after "A".
 - B.show() prints "B" after "C".
 - D.show() prints "D" after "B".
-

Key Points

- super() is mainly used to call parent methods/constructors.
 - In multiple inheritance, super() follows MRO to avoid duplicate calls.
 - It makes code **cleaner, reusable, and easier to maintain**.
 - It is often used in __init__ to ensure all parent constructors are executed.
-

MCQs on Inheritance in Python

1. Which of the following best defines inheritance in Python?

- a) Creating a new class from scratch
- b) Making a child class reuse the methods and attributes of a parent class

- c) Copying methods from one class to another
- d) Declaring variables inside a class

Answer: b

Explanation: Inheritance allows a child class to reuse parent class properties and methods.

2. What will be the output of the following code?

```
class Parent:
    def func1(self):
        print("Parent")
class Child(Parent):
    def func2(self):
        print("Child")
```

```
c = Child()
```

```
c.func1()
```

- a) Error
- b) Child
- c) Parent
- d) Parent Child

Answer: c

Explanation: func1() is inherited from Parent, so output is "Parent".

3. In Python, which symbol is used to define inheritance?

- a) : (colon)
- b) , (comma)
- c) () (parentheses)
- d) -> (arrow)

Answer: c

Explanation: Child class inherits using parentheses: class Child(Parent):.

4. Which of the following is NOT a type of inheritance in Python?

- a) Single
- b) Multiple
- c) Polymorphic
- d) Hybrid

Answer: c

Explanation: Polymorphism is an OOP concept, not a type of inheritance.

5. What will be the output of this code?

```
class A:  
    def show(self):  
        print("A")
```

```
class B(A):
```

```
    pass
```

```
class C(A):
```

```
    def show(self):  
        print("C")
```

```
class D(B, C):
```

```
    pass
```

```
d = D()
```

```
d.show()
```

- a) A
- b) B
- c) C
- d) Error

Answer: c

*Explanation: Due to **MRO (Method Resolution Order)**, class C's show() is called first.*

6. Which inheritance type is shown below?

```
class Father: pass
```

```
class Mother: pass
```

```
class Child(Father, Mother): pass
```

- a) Single
- b) Multiple
- c) Multilevel
- d) Hierarchical

Answer: b

Explanation: Child inherits from more than one parent.

7. In multilevel inheritance, if $A \rightarrow B \rightarrow C$, who is the grandchild class?

- a) A
- b) B
- c) C
- d) None

Answer: c

Explanation: C is the grandchild, inheriting through B from A.

8. Which of the following inheritance types has one parent and multiple children?

- a) Single
- b) Multiple
- c) Multilevel
- d) Hierarchical

Answer: d

Explanation: Hierarchical means one parent $\rightarrow$ multiple children classes.

9. What does MRO stand for in Python inheritance?

- a) Method Repetition Order
- b) Method Resolution Order
- c) Main Resource Object
- d) Multiple Resource Orientation

Answer: b

Explanation: MRO is Method Resolution Order, which decides method calling order in inheritance.

10. What will this code print?

```
class Parent:
```

```
    def greet(self):
```

```
        print("Hello from Parent")
```

```
class Child(Parent):
```

```
    def greet(self):
```

```
        print("Hello from Child")
```

```
c = Child()
```

```
c.greet()
```

- a) Hello from Parent
- b) Hello from Child
- c) Error
- d) None

Answer: b

Explanation: Child class overrides the method greet().

#	Question	Company	Year
1	What is inheritance?	TCS	2018
2	Explain single, multiple, multilevel, hierarchical & hybrid inheritance in Python.	Infosys	2019
3	What is Method Resolution Order (MRO)?	Wipro	2020
4	What happens if two parent classes have a method with the same name (in multiple inheritance)?	Amazon	2021
5	How do you call a parent class method from a child class?	Microsoft	2022
6	What is method overriding?	Google	2023
7	Can constructors be inherited? Explain with example.	IBM	2024
8	What is the use of super() in inheritance?	Cisco	2021
9	Explain the “diamond problem” and how Python resolves it.	Facebook	2022
10	Can you inherit from multiple classes and also override methods? Show sample code.	Apple	2023